PROJECT WIKI

Yumeniwa research

Research guides, written by Claude

Built 31 October 2026

CONTENTS

1 Research: How Hard Is “Bring Your Own Provider”? . . . 3
1.1 What BYO can mean: three levels . . . 3
1.2 The six hard parts . . . 3
1.3 How others did it . . . 4
1.4 Feasibility at a glance . . . 4
2 Research: Which Providers Can Yumeniwa Use? . . . 6
2.1 Five questions to ask of every provider . . . 6
2.2 Anime, manga and light novels . . . 6
2.3 Visual novels: VNDB is a tracker too . . . 7
2.4 Games: the hard case . . . 7
2.5 Google Drive backups . . . 9
2.6 The AniList outage of 21 October 2026 . . . 10
2.7 What must go through the backend . . . 10

CHAPTER 1

Research: How Hard Is "Bring Your Own Provider"?

RESEARCH GUIDE · WRITTEN BY CLAUDE

Facts and pointers gathered for me to study. It informs my design; it does not make my decisions. Researched 23 October 2026. Provider behaviour was checked with live requests from this laptop that day.

1.1 What BYO can mean: three levels

"Bring your own provider" is not one feature. It is a ladder, and each rung costs much more than the one below it.

Level	What the user brings	What Yumeniwa must already contain
1 · Configure	Credentials or a base URL for a provider **type** Yumeniwa already knows (their own AniList account, a self-hosted instance)	One adapter per provider type, written by you
2 · Describe	A file (JSON or similar) that describes an unknown API: endpoints, auth style, how its fields map onto Yumeniwa's fields	A generic engine that reads such files and a format (a "contract") that can express every API you want to support
3 · Code	A plugin: real code that talks to the provider	A plugin API, a loader, and a sandbox, because the code is untrusted

1.2 The six hard parts

1. Every provider authenticates differently

AniList and MyAnimeList use OAuth 2.0; VNDB uses a personal API token with permissions (`listread`, `listwrite`) (VNDB docs) others use API keys or nothing. A level-2 file must be able to describe all of these, including how tokens refresh.

2. Browsers block some providers (CORS)

An offline-first PWA wants to call providers from the browser. Whether it can depends on each provider's CORS headers. Preflight requests sent from origin `http://localhost:5173` on 23 Oct:

Provider	Result
AniList	Allowed: `Access-Control-Allow-Origin: *`
VNDB	Allowed: echoes the requesting origin, allows `Authorization`
MyAnimeList API	**Blocked**: preflight answered `405`, no CORS headers
Kitsu, MangaUpdates	Unclear: the preflight got `403`; test again with a real request

A provider that blocks browsers can only be reached through a server that forwards the request (a relay). So BYO decides part of what your Express backend is for.

3. A relay that calls user-supplied URLs is a security hole

If users can make your server call **any** URL, an attacker can point it at internal addresses. This attack is **server-side request forgery** (SSRF); read the OWASP SSRF prevention cheat sheet and KnoTex Ch 19 (SSRF).

4. Running someone else's code needs a real sandbox

Node's documentation states plainly: "The `node:vm` module is not a security mechanism" (Node docs). In the browser, isolation comes from Web Workers and sandboxed iframes (MDN: iframe `sandbox`). Level 3 is a security project of its own.

On a Linux server, isolation can also come from the operating system rather than the language: **seccomp** lets a process declare which system calls it may make, and the kernel kills it on any other. Cloudflare shows how to apply it without changing the program, through systemd's `SystemCallFilter=` or a preloaded library, and argues for allowlists over denylists and for killing the process over returning an error (Korchagin, "Sandboxing in Linux with zero lines of code", Cloudflare, 2020). It limits what a compromised plugin can do; it does not stop a plugin from misusing the network calls you allow it.

5. Providers disagree on meaning, not just format

Status sets differ (AniList has `REPEATING` and `PAUSED`; MyAnimeList has `on_hold`; VNDB uses labels), and score scales differ. A contract has to say what happens to a value that has no equivalent on the other side: this is **lossy mapping**. Concepts to read: **canonical data model** and the anti-corruption layer pattern.

6. Limits and maintenance

Each provider has its own rate limit (AniList 90 requests per minute, temporarily 30; VNDB 200 requests per 5 minutes), pagination and error format. And when a provider changes its API, every adapter or description file for it breaks: someone has to maintain them.

1.3 How others did it

Project	Approach
Mihon trackers	Six trackers built into the app (level 1); sync is one-way, app to tracker, and queued while offline
Mihon extensions	Level 3 for **content sources**: installable code packages from separate repositories, with trust warnings to the user
Home Assistant integrations	Level 3 with a strict contract: every integration ships a manifest, and a published quality scale grades them
Yamtrack, Ryot	No BYO: a fixed set of providers plus imports

1.4 Feasibility at a glance

- **Level 1** is ordinary adapter work: one module per provider behind one interface. Feasible from day one.
- **Level 2** needs a contract expressive enough for auth, pagination, rate limits and field mapping. Feasible, but designing that format well is a project on its own; study how others describe APIs (for example the OpenAPI specification) before inventing one.
- **Level 3** adds sandboxing and SSRF defences. High risk for a first version.

> **OPEN QUESTION** Questions for me to decide

> Which level does v1 need? Which providers ship built in? Can a provider that blocks CORS still work offline-first, and what does the relay mean for the “no single point of failure” goal?

CHAPTER 2

Research: Which Providers Can Yumeniwa Use?

RESEARCH GUIDE · WRITTEN BY CLAUDE

Facts and pointers gathered for me to study. It informs my design; it does not make my decisions. Researched 31 October 2026. Checked against each provider's official docs that evening. Items marked not re-checked come from research guide 01 (23 Oct) or general knowledge; look at them again before relying on them.

2.1 Five questions to ask of every provider

1. **Write or read?** Can Yumeniwa change the user's list on the provider, or only read from it?
2. **Auth:** how does the user let Yumeniwa act for them (OAuth, a personal token, an API key)?
3. **Limits:** how many requests, how fast?
4. **Terms:** non-commercial only? Attribution? A licence on the data?
5. **Browser:** can the PWA call it directly (CORS), or must the backend relay the request? Any secret that must never ship inside the PWA?

2.2 Anime, manga and light novels

Provider	Write to the list	Auth	Limits	Terms and browser
AniList	Yes: `SaveMediaListEntry` (GraphQL mutation)	OAuth 2.0: authorization code or implicit grant	90 requests/min; the docs currently warn the API is "in a degraded state and is limited to 30 requests per minute"	Browser: allowed (CORS `*`, 23 Oct). Light novels appear as format `NOVEL` (*not re-checked*).
MyAnimeList API v2	Yes: "Update my anime list status", "Update my manga list status" (`PATCH`), and delete	OAuth 2.0 with PKCE; "Currently, only the plain method is supported."	Not stated in the docs	Browser: **blocked** (preflight `405`, 23 Oct), so it needs the backend as a relay. Light novels are manga entries (*not re-checked*).
Kitsu	Yes: library entries (23 Oct)	OAuth 2.0 (*not re-checked*)	*Not checked*	API answered `200` on both `kitsu.io` and `kitsu.app` tonight. CORS unclear (23 Oct).
MangaUpdates	Yes: user lists; add or remove a series, update a list item (`/`	Bearer token from logging in	List writes can answer `412` "Five	Covers manga and novels. CORS unclear (23 Oct).

	`v1/lists/{id}/series`)		second update delay"	
MangaDex	Yes: `POST /manga/{id}/status` "Update Manga reading status" (reading, on_hold, plan_to_read, dropped, re_reading, completed); chapter read markers too	Only **personal clients** (password flow) exist today: "Only the account that owns a personal client can be used with it." Public clients are "not available yet".	*Not checked*	Usable by a single-user app with the user's own personal client; not by an app other people log in to. The API spec has no novel type: manga only.

2.3 Visual novels: VNDB is a tracker too

VNDB is not only a read-only database. It keeps a list for each user, and since 2023-01-17 its API can write that list:

- `PATCH /ulist/{id}`: "Add or update a visual novel in the user's list. Requires the `listwrite` permission." There is also `DELETE /ulist/{id}`, and `PATCH /rlist/{id}` for releases.
- **Auth:** a personal token the user creates on their profile ("Applications" tab, or `https://vndb.org/u/tokens`), with the permissions `listread` and `listwrite`, sent as `Authorization: Token ...`.
- **Limits:** "up to 200 requests per 5 minutes and up to 1 second of execution time per minute"; requests longer than 3 seconds are aborted.
- **Terms:** "free for non-commercial use"; the data falls under VNDB's Data License; the API is "provided on a best-effort basis".
- **Browser:** allowed (23 Oct).

2.4 Games: the hard case

> **WATCH OUT** **No game tracker with an official list-write API was found**
>
> Every games source checked is either **metadata only** or **read-only**. None lets Yumeniwa change a user's tracking status somewhere else.

Source	What it offers	Auth	Limits	Terms and browser
IGDB (Twitch)	Game metadata only; no user lists	Twitch app Client ID + Client Secret, `client_credentials` token	"4 requests per second", up to 8 open requests	"free for non-commercial usage under the terms of the Twitch Developer Service Agreement". Browsers "will run into CORS errors as the API does not allow requests directly from browsers", and the secret must never ship in a PWA: backend only.
RAWG	Game metadata only	API key	Free plan: "up to 20,000 requests per month"	Free plan: "Non-commercial projects only" and "Required backlinks to RAWG from pages where the data is used".

Steam Web API	Read-only: owned games and playtime (`IPlayerService/GetOwnedGames`)	Web API key + the user's SteamID	*Not checked*	Steam games only. Whether a private profile blocks it: *not re-checked.*
Backloggd, HowLongToBeat	No official API (23 Oct)	-	-	Scraping them would break their terms.

Gacha games: no tracker with an API was found.

What a game tracker actually records

For anime the progress is one number (episode 7 of 12). Game trackers record a different, richer shape:

What	Example from a real tracker	Source
Status	Backloggd has four toggles per game, Played, Playing, Backlog and Wishlist, which can be combined (a game can be Playing **and** Backlog). "Played" can be swapped for Completed, Abandoned, Retired or Shelved.	Backloggd About, changelog 1.4.0, 1.10 logging update
Diary of play	Backloggd's journal: a calendar where you mark the days you played. "Each log has its own journal", so one game can have several logs (playthroughs).	1.10 logging update
How far	HowLongToBeat's levels: Main Story, Main + Extras, Completionist (100%). The times are user-submitted and averaged.	unofficial wrapper's fields no official docs found
Hours played	Steam reports total playtime per owned game. Playnite counts the time of every game it launches, emulated ones included.	Valve wiki, Playnite
Achievements	Steam achievements, PlayStation trophies, Xbox achievements, RetroAchievements: a completion percentage per game.	See the next table
Ownership	Infinite Backlog records when and where you got a game, physical or subscription, even whether it is lent out (per a 2026 roundup, *not checked on its site*).	Achriom roundup
Rating, review	Common to all of them.	-

Where game data can be read from (none can be written to)

Source	What you can read	Auth	Caveats
Steam Web API (official)	Owned games with playtime (`GetOwnedGames`); achievements per game (`GetPlayerAchievements`)	Web API key + SteamID	`GetOwnedGames` works only "if the profile is publicly visible" (Valve wiki). Achievements on a private profile: *untested.*
RetroAchievements (official)	A user's completion progress for every game they played, with awards	Web API key from your RA control panel; "Do not expose your API key"	Retro and emulated games only. Read-only.

psn-api (PlayStation, **unofficial**)	Trophy lists, earned trophies per game, trophy summary	An NPSSO token taken from a browser login, which the docs call equivalent to your password	Not an official Sony API: it can break at any time, and handling a password-equivalent token is a security risk.
OpenXBL (Xbox, third-party)	Xbox achievements	Its own API key	Only a forum page calls it "free to use"; limits and terms *not confirmed.*
Backloggd, Grouvee, Infinite Backlog, HowLongToBeat	-	-	No official public API found. Community HowLongToBeat wrappers reverse-engineer the site and can break without notice.

Prior art worth reading: Playnite, an open-source (MIT) game library manager. It imports libraries from Steam, GOG, Epic and others through **plugins**, one per store, and keeps all library data locally. That is the same plugin-per-provider, local-first shape as Yumeniwa, applied to games.

> **OPEN QUESTION** **Questions for me (games)**
>
> - What is a game's **progress** in Yumeniwa: a status, hours, a completion level, an achievement percentage, a diary of sessions, or several of these?
> - Can one game have several playthroughs (Backloggd's several logs per game)? What does that do to the Record entity?
> - Since nothing can be written back, is every game record a manual record with metadata (IGDB/RAWG) and optional imports (Steam playtime, achievements) attached? Does that need a third record kind besides manual and provider?

2.5 Google Drive backups

Not like a Gmail app password. App passwords let old mail apps sign in with a password over IMAP or SMTP. The Drive API accepts only OAuth 2.0 access tokens, so Drive goes through a Google Cloud project:

1. Create a Google Cloud project and enable the Drive API.
2. Configure the OAuth consent screen.
3. Create an OAuth client ID of type "Web application", listing your app's origins.
4. Request an access token with a scope, then call the API with it (Google OAuth 2.0 overview).

Scopes (Choose Drive API scopes):

- `drive.appdata`: a hidden folder only your app sees ("View and manage the app's own configuration data in your Google Drive"). **Non-sensitive.**
- `drive.file`: only files your app created or the user picked. **Non-sensitive.**
- `drive`: every file. **Restricted:** it needs restricted-scope verification, and "If you store restricted scope data on servers (or transmit), then you must go through a security assessment."
- Google's advice: "choose the most narrowly focused scope possible."

Verification (Google Cloud help): with only non-sensitive scopes, "it is not mandatory for your app to complete the app verification process." Brand verification is needed only to show an app name and logo on the consent screen.

Two catches:

- While the project's publishing status is "Testing" (external users), refresh tokens expire after 7 days, so the user must consent again every week. Publishing to "In production" with non-sensitive scopes avoids this.
- A browser-only PWA using Google Identity Services' token model gets short-lived access tokens and must call `requestAccessToken()` again "from a user-driven event" (such as a button press). So no silent background backups from the browser alone; those need the authorization code model, with the backend holding a refresh token.

How hard: the console setup is about half an hour of forms. The OAuth flow is the same kind you already need for AniList and MyAnimeList. The real work is the choices above: which scope, and whether backups run in the background.

2.6 The AniList outage of 21 October 2026

- **What AniList's docs say** (Considerations): in severe outages AniList may lower rate limits or "temporarily suspend access to the API", announced on its Discord. A suspended API answers `403` with "The AniList API has been temporarily disabled due to severe stability issues. Please check the official AniList Discord for more information."
- **The date:** confirmed only by third-party projects that reacted to it on 2026-10-21 (for example Aaru PR 86, AniShift issue 64). I found no dated AniList page.
- **The PHP flaw is real:** "PHP2Shell", CVE-2026-109618 and CVE-2026-109619, unauthenticated remote code execution in PHP core. PHP shipped emergency fixes on 21 October (8.2.35, 8.3.36, 8.4.27, 8.5.12); attacks began on 17 October; CISA added both to its Known Exploited Vulnerabilities catalog (SANS ISC, BleepingComputer).
- **The link between the two is not sourced.** No source found says AniList's shutdown was caused by PHP2Shell; only the dates match. To keep that claim in the wiki, find AniList's Discord announcement. Otherwise state the two facts separately, without the cause.

2.7 What must go through the backend

From the facts above:

- **Blocked in the browser:** MyAnimeList (CORS) and IGDB (CORS, and a secret).
- **Background Drive backups:** need a refresh token held by the backend.
- **Callable from the browser:** AniList and VNDB.
- **Unclear:** Kitsu and MangaUpdates (test with a real request).

> **OPEN QUESTION** **Questions for me to decide**
>
> - Which provider covers each v1 media type? One per type, or several for anime and manga?
> - Games: if no tracker can be written to, is Yumeniwa itself the tracker for games, with IGDB or RAWG for metadata and Steam for playtime imports? Or do games leave v1?
> - Drive (decided 31 Oct: after v1, before v2): which scope? Is a "Back up now" button enough, or are background backups needed (which means the backend)?
> - MyAnimeList and IGDB need the backend as a relay. What does that do to offline-first and to "no single point of failure"?
> - The non-commercial terms (VNDB, IGDB, RAWG): does that matter for how you publish Yumeniwa?